

Constraint Exercise 2 slide 137:

First File: Package (my_pkg)

1. Create a package named my_pkg.
2. Declare three parameters:
 - HEIGHT = 10
 - WIDTH = 10
 - PERCENT_WHITE = 20
3. Define an enumerated type colors_t with two values:
 - BLACK
 - WHITE
4. Create a class named screen:
 - Declare a 2D randomized unpacked array pixels of type colors_t, with dimensions [HEIGHT][WIDTH].
 - Define a constraint block (colors_c) to enforce a distribution:
 - WHITE should appear PERCENT_WHITE% of the time.
 - BLACK should appear (100 - PERCENT_WHITE)% of the time.
 - Implement a print_screen function (void):
 - Iterates over pixels.
 - Prints each row of the array.

#Second File: Module (image)

1. Import the package (my_pkg).
2. Create a screen class object.
3. Use an initial block to:
 - Randomize the object.
 - Call print_screen() to display the 2D array.

Functional Coverage Exercise 1 slide 157:

First File: Package (testing_pkg)

1. Create a package named testing_pkg.
 2. Define a typedef enum, and class with rand variables and covergroup: (check notes)
 - Use @(posedge clk) as the sampling event.
 3. Implement a constructor to create the covergroup instance inside the class.
- >> Import the package in the ALU design file

Second File: Module (tb)

1. Create an object of the class (from testing_pkg) - import the package
2. Declare stimulus and response signals.
3. Generate the clock.
4. Instantiate the DUT.
5. Inside an initial block:
 - Reset the design.
 - Use a repeat (32) loop:
 - Randomize the object.
 - Pass randomized values to the DUT at each negedge clk.

Functional Coverage Exercise 2 slide 163:

- Add cross coverage with 2 options, weight set to 5 and cross_auto_bin_max to zero
- Add 2 bins to the cross coverage, crossing the max pos with the add or sub coverpoint and crossing the max neg with the add or sub coverpoint